Fiche de défense — RAG against the machine

Retrieval-Augmented Generation sur le codebase vLLM • Python 3.10 • uv • BM25 • Qwen3-0.6B

1. Le projet en une phrase

Un système **RAG** qui répond à des questions sur le code source de **vLLM** : il **indexe** le repo, **retrouve** les morceaux pertinents avec BM25, et **génère** une réponse avec Qwen3-0.6B en s'appuyant sur ces morceaux.

2. Le pipeline RAG (les 4 étapes)

Indexing : structurer les données pour les rendre cherchables. **Retrieving** : encoder la question, chercher par similarité, classer. **Augmenting** : insérer les morceaux récupérés dans le contexte du LLM. **Generating** : le LLM lit le contexte et produit la réponse.

3. Les 6 commandes CLI (Python Fire)

```
index           build + sauvegarde l'index BM25 depuis data/raw/vllm-0.10.1
search          une requête ad-hoc -> top-k MinimalSource
search_dataset  batch : 100 questions JSON -> StudentSearchResults
answer          une question -> retrieval + génération Qwen
answer_dataset  batch : génère une réponse par search result
evaluate        recall@k des résultats vs ground truth
```

4. Architecture (src/student/)

```
__main__.py   CLI Fire, 6 sous-commandes (orchestration pure)
models.py     pydantic (MinimalSource, ...) + Chunk interne
chunking.py   walk_files + 3 stratégies + chunk_file (dispatcher)
tokenizer.py  tokenize() : texte -> tokens (index ET requête)
indexer.py    build / save / load index BM25 + search()
generator.py  QwenGenerator : charge le modèle 1x, answer()
evaluator.py  recall@k
```

Le module DOIT s'appeler `student` : le sujet l'appelle via `python -m student` (V.6.2).

5. Chunking — 3 stratégies (max 2000 char, configurable)

Python (.py) : module `ast`, 1 chunk par fonction/classe top-level (fallback fenêtre fixe si SyntaxError). **Markdown (.md)** : découpe par titre ATX (#..######) via regex. **Fallback** : fenêtre glissante de N caractères avec overlap (sous-chunke aussi tout chunk trop gros). Chaque chunk garde file_path + indices caractères **relatifs au fichier d'origine**.

6. BM25 — la méthode de retrieval

Algo lexical qui classe chaque chunk par **TF** (fréquence du mot, avec saturation k1=1.5) × **IDF** (rareté du mot), normalisé par la longueur (b=0.75). Améliore TF-IDF sur 2 points : saturation du TF et normalisation longueur. **Limite** : purement lexical (pas de sémantique) — d'où les bonus embeddings/hybride.

7. Tokenisation — le levier de performance

Identifiant-aware : split snake_case et camelCase (`get_supported_mm_limits -> get supported mm limits`) + lowercase. C'est ce qui fait matcher une question en langage naturel avec un nom de fonction.
Stopwords asymétriques : l'index garde TOUT (IDF correct), mais les REQUÊTES retirent les mots grammaticaux (how, what, the, is...). Les questions sont verbeuses : ça épure le ranking. **0.78 -> 0.80 sur docs.**

8. recall@k et le "k"

k = nombre de résultats demandés (top-k). **recall@5** = la bonne source est-elle dans les 5 premiers ? Une source est "trouvée" si ≥ **5% d'overlap caractères** avec une source du ground truth. Plusieurs sources : score = trouvées / total. Recall global = moyenne sur toutes les questions. Évalué à k=5.

9. Génération — Qwen3-0.6B

`transformers` : AutoTokenizer + AutoModelForCausalLM. Device auto (cuda/mps/cpu). **Chat template** (ChatML) obligatoire car Qwen est conversationnel. `do_sample=False` = déterministe. `enable_thinking=False` = pas de bloc <think>, réponse directe. Chargé **une seule fois** (coût ~10s, hors boucle).

10. Les chiffres (à connaître par cœur)

	Mesuré	Seuil sujet

Recall@5 docs	0.80	0.80
Recall@5 code	0.54	0.50
Indexing (26k chunks)	~5 s	<= 5 min
Cold start (modèle)	~10-15 s	<= 60 s
Réglage retenu	max_chunk_size=1000	—

11. Choix techniques à savoir justifier

max_chunk_size=1000 : sweet spot. Plus petit = meilleure précision docs mais code fragmenté ; plus grand = l'inverse.
type: ignore (2) : un sur l'override pydantic prescrit par le sujet (invariance des listes), un sur model.generate (typage lâche de transformers). Marche au runtime.
cast(str, ...) : apply_chat_template/decode renvoient une Union ; tokenize=False garantit un str.
chunks.json : BM25 ne stocke que des indices ; ce fichier mappe indice -> (file_path, start, end) pour reconstruire les MinimalSource.

12. Questions pièges probables

Q. Pourquoi BM25 et pas des embeddings ?

Le sujet impose TF-IDF ou BM25 en mandatory. BM25 est très fort sur les identifiants/mots techniques exacts, rapide, sans GPU. Les embeddings sont un bonus (sémantique).

Q. Différence chunk vs MinimalSource ?

Chunk = MinimalSource + le texte. Le texte sert au LLM ; le MinimalSource (file_path + indices) est le format de sortie.

Q. Pourquoi sauver l'index sur disque ?

Pour respecter le cold start <= 60s : on charge (~200ms) au lieu de reconstruire à chaque fois.

Q. Comment améliorer encore le recall ?

Index séparés docs/code (autorisé V.8), embeddings + retrieval hybride, query expansion (bonus chap. VIII).

Q. Pourquoi trier les fichiers à l'indexation ?

Reproductibilité : sans tri, l'ordre dépend du système de fichiers et les indices de chunks changeraient.

13. Relancer le jour J

```
make install
uv run python -m student index --max_chunk_size 1000
make lint
uv run python -m student search "How to configure the OpenAI server?" --k 10
```